

Matching & Mapping Data - Schema

Vu Tuyen Trinh
SoICT - HUST

1

Schema Matching and Mapping



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

63

63

Problem

- We have described
 - formalisms to specify source descriptions
 - algorithms that use these descriptions to reformulate queries
- How to create the source descriptions?
 - often begin by creating semantic matches
 - name = title, location = concat(city, state, zipcode)
 - then elaborate matches into semantic mappings
 - e.g., structured queries in a language such as SQL
- Schema matching and mapping are often quite difficult
- This chapter describes matching and mapping tools
 - that can significantly reduce the time it takes for the developer to create matches and mappings

Techniques

- Schema matching
 - Matchers
 - Combining match predictions
 - Enforcing domain integrity constraints
 - Match selector
 - Reusing previous matches
 - Many-to-many matches
- Schema mapping

Semantic Mappings

DVD-VENDOR

Movies(id, title, year)

Products(mid, releaseDate, releaseCompany, basePrice, rating, saleLocID)

Locations(lid, name, taxRate)

AGGREGATOR

Items(name, releaseInfo, classification, price)

- Let S and T be two relational schemas
 - refer to the attributes and tables of S and T as their elements
- A semantic mapping is a query expression that relates a schema S with a schema T
 - the following mapping shows how to obtain Movies.title
 - SELECT name as title
FROM Items

Semantic Mappings

DVD-VENDOR

Movies(id, title, year)

Products(mid, releaseDate, releaseCompany, basePrice, rating, saleLocID)

Locations(lid, name, taxRate)

AGGREGATOR

Items(name, releaseInfo, classification, price)

- More examples of semantic mappings
 - the following mapping shows how to obtain Items.price
 - SELECT (basePrice * (1 + taxRate)) AS price
FROM Products, Locations
WHERE Products.saleLocID = Locations.lid
 - the following mapping shows how to obtain an entire tuple for Items table of AGGREGATOR
 - SELECT title AS name, releaseDate AS releaseInfo, rating AS classification,
basePrice * (1 + taxRate) AS price
FROM Movies, Products, Locations
WHERE Movies.id = Products.mid AND Products.saleLocID =
Locations.lid

Example

- Consider building a DI system
 - over two sources, with schemas DVD-VENDOR & BOOK-VENDOR
 - assume the mediated schema is AGGREGATOR
- If we use Global-as-View approach to relate schemas
 - must describe Items in AGGREGATOR as a query over sources
 - to do this, create semantic mappings m_1 and m_2 that specify how to obtain tuples of Items from DVD-VENDOR and BOOK-VENDOR, respectively, then return semantic mapping $(m_1 \text{ UNION } m_2)$ as the GAV description of Items table.

Example (cont.)

- If we use Local-as-View approach to relate schemas
 - for each table in DVD-VENDOR and BOOK-VENDOR, must create a semantic mapping that specifies how to obtain tuples for that table from schema AGGREGATOR (i.e., from table Items)
- If we use GLAV approach
 - there are semantic mappings going in both directions

DVD-VENDOR
Movies(id, title, year)
Products(mid, releaseDate, releaseCompany, basePrice, rating, saleLocID)
Locations(lid, name, taxRate)

AGGREGATOR
Items(name, releaseInfo, classification, price)

Semantic Matches

- A semantic match relates a set of elements in a schema S to a set of elements in schema T
 - without specifying in detail (to the level of SQL queries) the exact nature of the relationship (as in semantic mappings)
- One-to-one matches
 - $\text{Movies.title} = \text{Items.name}$
 - $\text{Products.rating} = \text{Items.classification}$
- One-to-many matches
 - $\text{Items.price} = \text{Products.basePrice} * (1 + \text{Locations.taxRate})$
- Other types of matches
 - many-to-one, many-to-many

Relationship between Schema Matching and Mapping

- To create source description
 - often start by creating semantic matches
 - then elaborate matches into mappings
- Why start with semantic matches?
 - they are often easier to elicit from designers
 - e.g., can specify $\text{price} = \text{basePrice} * (1 + \text{taxRate})$ from domain knowledge
- Why the need to elaborate matches into mappings?
 - matches often specify functional relationships
 - but they cannot be used to obtain data instances
 - need SQL queries, that is, mappings for that purpose
 - so matches need to be elaborated into mappings

Schema Matching and Mapping

- Example: elaborate the match
 - $\text{price} = \text{basePrice} * (1 + \text{taxRate})$into mapping
 - `SELECT (basePrice * (1 + taxRate)) AS price`
`FROM Product, Location`
`WHERE Product.saleLocID = Location.lid`
- Another reason for starting with matches
 - break the long process in the middle
 - allow designer to verify and correct the matches
 - thus reducing the complexity of the overall process

Challenges

- Matching and mapping systems must reconcile semantic heterogeneity between the schemas
- Such semantic heterogeneity arise in many ways
 - same concept, but different names for tables and attributes
 - rating vs classification
 - multiple attributes in 1 schema relate to 1 attribute in the other
 - basePrice and taxRate relate to price
 - tabular organization of schemas can be quite different
 - one table in AGGREGATOR vs three tables in DVD-VENDOR
 - coverage and level of details can also differ significantly
 - DVD-VENDOR also models releaseDate and releaseCompany

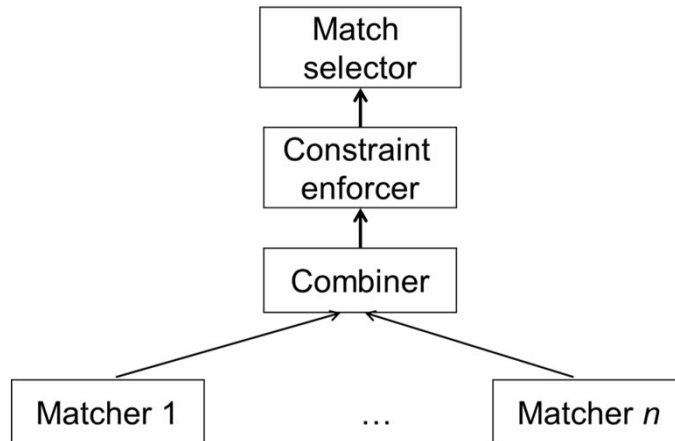
Challenges (cont.)

- Why do we have semantic heterogeneity?
 - schemas are created by different people whose states and styles are different
 - disparate databases are rarely created for exact same purposes
- Why reconciling semantic heterogeneity is hard
 - the semantics is not fully captured in the schemas
 - schema clues can be unreliable
 - intended semantics can be subjective
 - correctly combining the data is difficult
- Standard is not a solution!
 - works for limited use cases where number of attributes is small and there is strong incentive to agree on them

Overview of Matching Systems

- For now we consider only 1-1 matching systems
 - will discuss finding complex matches later
- Key observation: need multiple heuristics / types of information to maximize matching accuracy
 - e.g., by matching the names, can infer that releaseInfo = releaseDate or releaseInfo = releaseCompany, but do not know which one
 - by matching the data values, can infer that releaseInfo = releaseDate or releaseInfo = year, but do not know which one
 - by combining both, can infer that releaseInfo = releaseDate

Matching System Architecture



76

Overview of Mapping Systems

- Input: matches, output: actual mappings
- Key challenge: find how tuples from one source can be transformed and combined to produce tuples in the other
 - which data transformation to apply?
 - which joins to take?
 - and many more possible decisions

77

Outline

- Problem definition, challenges, and overview
- Schema matching
 - Matchers
 - Combining match predictions
 - Enforcing domain integrity constraints
 - Match selector
 - Reusing previous matches
 - Many-to-many matches
- Schema mapping

Matchers

- schemas → similarity matrix
- Input: two schemas S and T , plus any possibly helpful auxiliary information (e.g., data instances, text descriptions)
- Output: sim matrix that assigns to each element pair of S and T a number in $[0,1]$ predicting whether the pair match
- Numerous matchers have been proposed
- We describe a few, in two classes: name matchers and data matchers

Name-Based Matchers

- Use string matching techniques
 - e.g., edit distance, Jaccard, Soundex, etc.
- Often have to pre-process names
 - split them using certain delimiters
 - e.g., saleLocID → sale, Loc, ID
 - expand known abbreviations or acronyms
 - loc → location, cust → customer
 - expand a string with synonyms / hypernyms
 - add cost to price, expand product into book, dvd, cd
 - remove stop words
 - in, at, and

Example

DVD-VENDOR

Movies(id, title, year)

Products(mid, releaseDate, releaseCompany, basePrice, rating, saleLocID)

Locations(lid, name, taxRate)

AGGREGATOR

Items(name, releaseInfo, classification, price)

name-base matcher:

name = <name: 1, title: 0.2>

releaseInfo = <releaseDate: 0.5, releaseCompany: 0.5>

price = <basePrice: 0.8>

Instance-Based Matchers

- When schemas come with data instances, these can be extremely helpful in deciding matches
- Many instance-based matchers have been proposed
- Some of the most popular
 - recognizers
 - use dictionaries, regexes, or simple rules
 - overlap matchers
 - examine the overlap of values among attributes
 - classifiers
 - use learning techniques

Building Recognizers

- Use dictionaries, regexes, or rules to recognize data values of certain kinds of attributes
- Example attributes for which recognizers are well suited
 - country names, city names, US states
 - person names (can use dictionaries of last and first names)
 - color, rating (e.g., G, PG, PG-13, etc.), phone, fax, soc sec
 - genes, protein, zip codes

Measuring the Overlap of Values

- Typically applies to attributes whose values are drawn from some finite domain
 - e.g., movie ratings, movie titles, book titles, country names
- Jaccard measure is commonly used
- Example:
 - use Jaccard measure to build a data-based matcher between DVD-VENDOR and AGGREGATOR
 - AGGREGATOR.name refers to DVD titles, DVD-VENDOR.name refers to sale locations, DVD-VENDOR.title refers to DVD titles
 - ➔ low score for (name, name), high score for (name, title)

data-based matcher:

name = <name: 0.2, title: 0.5>

releaseInfo = <releaseDate: 0.7>

classification = <rating: 0.6>

price = <basePrice: 0.2>



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

84

84

Using Classifiers

- Builds classifiers on one schema and uses them to classify the elements of the other schema
 - e.g., use Naïve Bayes, decision tree, rule learning, SVM
- A common strategy
 - for each element s_i of schema S , want to train classifier C_i to recognize instances of s_i
 - to do this, need positive and negative training examples
 - take all data instances of s_i (that are available) to be positive examples
 - take all data instances of other elements of S to be negative examples
 - train C_i on the positive and negative examples



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

85

85

Using Classifiers

- A common strategy (cont.)
 - now we can use C_i to compute sim score between s_i and each element t_j of schema T
 - to do this, apply C_i to data instances of t_j
 - for each instance, C_i produces a number in $[0,1]$ that is the confidence that the instance is indeed an instance of s_i
 - now need to aggregate the confidence scores of the instances (of t_j) to return a single confidence score (as the sim score between s_i and t_j)
 - a simple way to do so is to compute the average score over all instances of t_j

Using Classifiers: An Example

- s_i is address, t_j is location
- Sim scores are 0.9, 0.7, and 0.5, respectively for the three instances of T.location → return average score of 0.7 as sim score between address and location

SCHEMA S

current-showing	address	phone
Lord of the Rings	Madison WI	(608) 695 2311
	Mountain View CA	(650) 277 1358

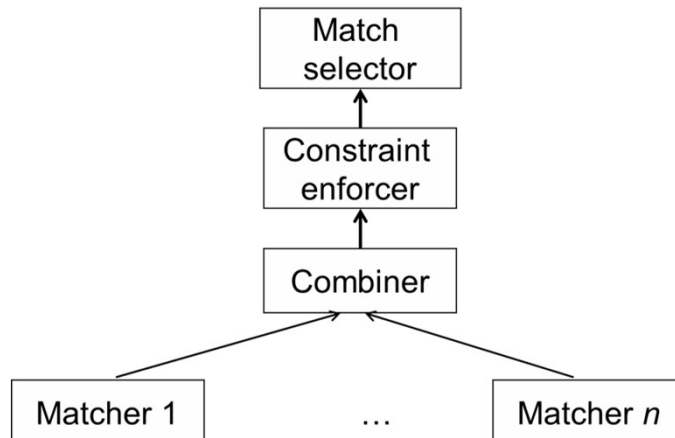
SCHEMA T

name	location	phone
...	Milwaukee WI	...
	Palo Alto CA	
	Philadelphia PA	

Using Classifiers

- Designer decides which schema should play the role of schema S (on which to build classifiers)
 - typically chooses the mediated schema to be S, so that can reuse the classifiers to match the schemas of new data sources
- May want to do it both ways
 - build classifiers on S and use them to classify instances of T
 - then build classifiers on T and use them to classify instances of S
 - e.g., when both S and T are taxonomies of concepts

Reminder: Matching System Architecture



Combining Match Predictions

- A combiner merges the sim matrices output by the matchers into a single matrix
- Simplest types of combiner return average, minimum, or maximum of scores
- For example, an average combiner computes the score between s_i and t_j to be
 - $\text{combined}(i,j) = [\sum_{m=1}^k \text{matcherScore}(m, i, j)]/k$
 - here k matchers are used
 - $\text{matcherScore}(m, i, j)$ is score between s_i and t_j as produced by the m -th matcher



TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

90

90

Combining Match Predictions - Example

DVD-VENDOR

Movies(id, title, year)

Products(mid, releaseDate, releaseCompany, basePrice, rating, saleLocID)

Locations(lid, name, taxRate)

AGGREGATOR

Items(name, releaseInfo, classification, price)

name-based matcher:

name=<name: 1, title: 0.5>

releaseInfo=<releaseDate:0.5, releaseCompany: 0.5>

price =<basePrice:0.8>

data-based matcher:

name = <name: 0.2, title: 0.5>

releaseInfo = <releaseDate: 0.7>

classification = <rating: 0.6>

price = <basePrice: 0.2>

average-combiner:

name = <name: 0.6, title: 0.5>

releaseInfo = <releaseDate: 0.6, releaseCompany: 0.25>

classification = <rating: 0.3>

price = <basePrice: 0.5>



TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

91

91

Combining Match Predictions

- When to use which combiner?
 - average combiner : when we do not have any reason to trust one matcher over the others
 - maximum combiner: when we trust a strong signal from matchers, i.e., if a matcher outputs a high value, we are relatively confident that the two elements match
 - minimum combiner: when we want to be more conservative
- More complex types of combiners
 - use hand-crafted scripts
 - e.g., if s_i is address, return the score of the data-based matcher
 - otherwise, return the average score of all matchers



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

92

92

Combining Match Predictions

- More complex types of combiners (cont.)
 - weighted-sum combiners
 - give weights to each matcher, according to its importance
 - may learn the weights from training data
 - can combine the weights in many ways: linear regression, logistic regression, etc.
 - the combiner itself can be a learner, which learns how to combine the scores of the matchers
 - e.g., decision tree, logistic regression, etc.

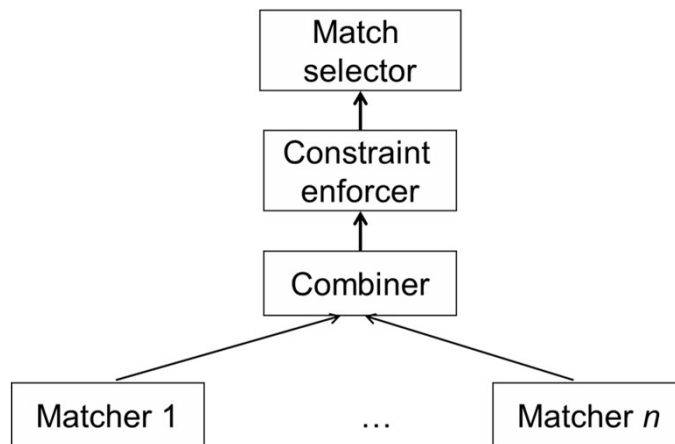


SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

93

93

Reminder: Matching System Architecture



Enforcing Domain Integrity Constraints

- Designer often has knowledge that can be naturally expressed as domain integrity constraints
- Constraint enforcer exploits these to prune certain match combinations
 - searches through the space of all match combinations produced by the combiner
 - finds one combination with the highest aggregated confidence score that satisfies the constraints

Illustrating Example

- Here we have four match combinations $M_1 - M_4$
 - $M_1 = \{\text{name} = \text{name}, \text{releaseInfo} = \text{releaseDate}, \text{classification} = \text{rating}, \text{price} = \text{basePrice}\}$
 - For each M_i , can compute an aggregated score
 - e.g., by multiplying the individual scores,
so $\text{score}(M_1) = 0.6 * 0.6 * 0.3 * 0.5$

average-combiner:

name = <name: 0.6, title: 0.5>

releaseInfo = <releaseDate: 0.6, releaseCompany: 0.25>

classification = <rating: 0.3>

price = <basePrice: 0.5>



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

96

96

Illustrating Example (Cont.)

- Suppose designer knows that
 - AGGREGATOR.name refers to movie titles
 - many movie titles contain at least four words
- Designer can specify a constraint such as
 - if an attribute A matches AGGREGATOR.name, then in any random sample of 100 data values of A, at least 10 values must contain four words or more
- Now the constraint enforcer can search for the best match combination that satisfies this constraint



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

97

97

Illustrating Example (Cont.)

- How to search?
 - conceptually, check the combination with the highest score, M_1 : it does not satisfy the constraint
 - check the combination with the next highest score, M_2 :
this one satisfies the constraint, so return it as the desired match combination
 - name = title, releaseInfo = releaseDate, classification = rating, price = basePrice
- In practice exploiting constraints is quite hard
 - must handle a variety of constraints
 - must find a way to search efficiently

Domain Integrity Constraints

- Two kinds of constraints: hard and soft
- Hard constraints
 - must be enforced
 - no output match combination can violate them
- Soft constraints
 - of more heuristic nature, may actually be violated
 - we try to minimize the degree to which extent these constraints are violated
- Each constraint is associated with a cost
 - for hard constraints, the cost is ∞
 - for soft constraints, the cost can be any positive number

Example

BOOK-VENDOR

Books(ISBN, publisher, pubCountry, title, review)

Inventory(ISBN, quantity, location)

DISTRIBUTOR

Items(code, name, brand, origin, desc)

InStore(code, availQuant)

	Constraints	Costs
c_1	If $A = \text{Items.code}$, then A is a key	∞
c_2	If $A = \text{Items.desc}$, then any random sample of 100 data instances of A must have an average length of at least 20 words	1.5
c_3	If $A_1 = B_1$, $A_2 = B_2$, B_2 is next to B_1 in the schema, but A_2 is not next to A_1 , then there is no A^* next to A_1 such that $ \text{sim}(A^*, B_2) - \text{sim}(A_2, B_2) \leq t$ for a small pre-specified t	2
c_4	If more than half of the attributes of Table U match those of Table V , then $U = V$	1

100

Domain Integrity Constraints

- Each constraint is specified only once by the designer
- Key requirement
 - given a constraint c and a match combination M , the enforcer must be able to efficiently decide whether M violates c , given all the available data instances of the schemas
- If the enforcer cannot detect a violation, that does not mean that the constraint indeed holds, may just mean that there is not enough data to verify
 - e.g., if all current data instances of A are distinct, that does not mean A is a key

101

Searching the Space of Match Combinations

- There are many ways to do this, depending on the application and the types of constraints involved
- We describe here two methods
 - an adaptation of A^* search
 - guaranteed to find the optimal solution
 - but computationally more expensive
 - local propagation
 - faster
 - but performs only local optimizations

Review: A^* Search

- A^* searches for a goal state within a set of states, beginning from an initial state
- Each path through the search space is assigned a cost
- A^* finds the goal state with the cheapest path from the initial state
- Performs best-first search
 - starts with the initial state, expand this state into a set of states
 - selects the state with the smallest estimated cost
 - expands the selected state into a set of states
 - again selects the state with the smallest estimated cost, etc.

Review: A* Search

- Estimated cost of a state n is $f(n) = g(n) + h(n)$
 - $g(n)$ = cost of path from initial state to n
 - $h(n)$ = a lower bound on cost from n to a goal state
 - $f(n)$ = a lower bound on the cost of the cheapest solution via n
- A* terminates when reaching a goal state, returning path
 - guaranteed to find a solution if exists, and the cheapest one

Applying Constraints with A* Search

- Goal: apply A* to match schemas S_1 and S_2
 - S_1 has attributes $A_1, \dots, A_n$
 - S_2 has attributes $B_1, \dots, B_m$
- A state = a tuple of size n
 - the i -th element either specifies a match for A_i , or a wildcard $*$, representing that the match for A_i is yet undetermined
 - a state can be viewed as a set of match combinations that are consistent with the specifications
 - e.g., $(B_2, *, B_1, B_3, B_2)$
 - a state is abstract if it contains wildcards, is concrete otherwise

Applying Constraints with A* Search

- Initial state: $(*, *, \dots, *)$ = all match combinations
- Goal states: those that do not contain any $*$
- Expanding states:
 - can only expand an abstract state
 - choose a $*$ and replace it with all possible matches
 - a key decision is which $*$ to expand

Applying Constraints with A* Search

- Cost of goal states:
 - combines our estimate of the likelihood of the combination and the degree to which it violates the constraints
 - $\text{cost}(M) = -\text{LH}(M) + \text{cost}(M, c_1) + \dots + \text{cost}(M, c_p)$
 - $\text{LH}(M)$ = likelihood of M according to the sim matrix
 $= \log \text{conf}(M)$
 - if $M = (B_{k1}, \dots, B_{kn})$ then
 $\text{conf}(M) = \text{combined}(1, k_1) * \dots * \text{combined}(1, k_n)$
 - $\text{cost}(M, c_i)$ the degree to which M violates constraint c_i
- Cost of abstract states:
 - estimating this is quite involved, using approximation over the unknown wildcards (see notes)

Applying Constraints with Local Propagation

- Propagate constraints locally from schema elements to their neighbors until we reach a fixed point
- First select constraints that involve element's neighbors
- Then rephrase them to work with local propagation

	Constraints	Costs
c_1	If $A = \text{Items.code}$, then A is a key	∞
c_2	If $A = \text{Items.desc}$, then any random sample of 100 data instances of A must have an average length of at least 20 words	1.5
c_3	If $A_1 = B_1, A_2 = B_2, B_2$ is next to B_1 in the schema, but A_2 is not next to A_1 , then there is no A^* next to A_1 such that $ \text{sim}(A^*, B_2) - \text{sim}(A_2, B_2) \leq t$ for a small pre-specified t	2
c_4	If more than half of the attributes of Table U match those of Table V , then $U = V$	1



108

108

An Example

- rephrasing c_3 :

	Constraints	Costs
c_1	If $A = \text{Items.code}$, then A is a key	∞
c_2	If $A = \text{Items.desc}$, then any random sample of 100 data instances of A must have an average length of at least 20 words	1.5
c_3	If $A_1 = B_1, A_2 = B_2, B_2$ is next to B_1 in the schema, but A_2 is not next to A_1 , then there is no A^* next to A_1 such that $ \text{sim}(A^*, B_2) - \text{sim}(A_2, B_2) \leq t$ for a small pre-specified t	2
c_4	If more than half of the attributes of Table U match those of Table V , then $U = V$	1



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

109

109

Local Propagation Algorithm

- Initialization:

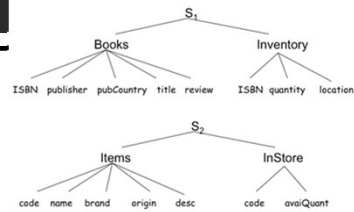
- represent S_1 and S_2 as graphs
- algorithm computes a sim matrix SIM which is initialized to be the combined result of S_1 and S_2 (using a combiner)

- Iteration:

- select a node s_1 in graph of S_1 , update the values in SIM based on similarities computed for its neighbors
- if perform tree traversal, go bottom-up, starting from the leaves

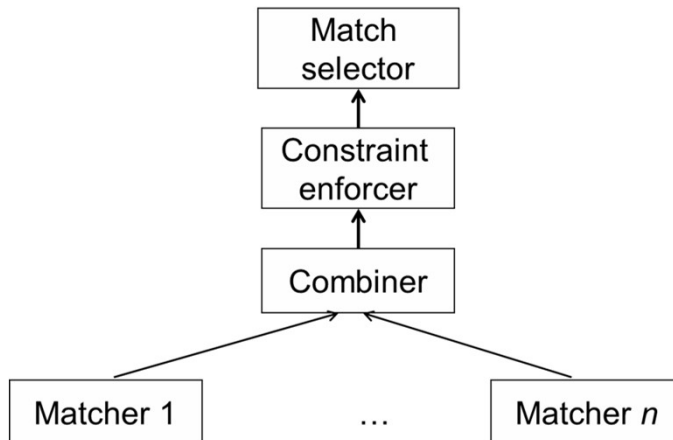
- Termination:

- after either a fixed number of iterations or when the changes to SIM are smaller than a pre-defined threshold



110

Reminder: Matching System Architecture



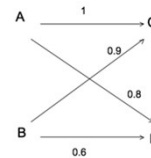
111

Match Selector

- Selects matches from the sim matrix
- Simplest strategy: thresholding
 - all attribute pairs with sim not less than a threshold are returned as matches
 - e.g., given the matrix
 - name = <title: 0.5>
 - releaseInfo = <releaseDate: 0.6>
 - classification = <rating: 0.3>
 - price = <basePrice: 0.5>
 given threshold 0.5, return matches name = title, etc.
- More complex strategies return the top few match combinations

A Common Strategy to Select a Match Combination

- Elements of S = men, elements of T = women
- $\text{sim}(i,j)$ = the degree to which A_i and B_j desire each other
- Find a stable match combination between men and women
- A match combination would be unstable if
 - there are two couples $A_i = B_j$ and $A_k = B_l$ such that A_i and B_l want to be with each other, i.e., $\text{sim}(i,l) > \text{sim}(i,j)$ and $\text{sim}(i,l) > \text{sim}(k,l)$
- Other algorithms exist to select a match combination



Outline

- Problem definition, challenges, and overview
- Schema matching
 - Matchers
 - Combining match predictions
 - Enforcing domain integrity constraints
 - Match selector
 - Reusing previous matches
 - Many-to-many matches
- Schema mapping

Reusing Previous Matches

- Schema matching tasks are often repetitive
 - e.g., keep matching new sources into the mediated schema
- Can a schema matching system improve over time? Can it learn from previous experience?
- Yes, one way to do this is to use machine learning techniques
 - consider matching sources $S_1, \dots, S_n$ into a mediated schema G
 - we manually match $S_1, \dots, S_m$ into G (where $m \ll n$)
 - the system generalizes from these matches to predict matches for $S_{m+1}, \dots, S_n$
 - use a technique called multi-strategy learning

Multi-Strategy Learning: Training Phase

- Employ a set of learners $L_1, \dots, L_k$
 - each learner creates a classifier for an element e of the mediated schema G , from training examples of e
 - these training examples are derived using semantic matches between the training sources $S_1, \dots, S_m$ and G
- Use a meta-learner to learn a weight w_{e,L_i} for each element e of the mediated schema and each learner L_i
 - these weights will be used later in the matching phase to combine the predictions of the learners L_i

Example of Training Phase

- Mediated schema G has three attributes: e_1, e_2, e_3
- Use two learners: Naive Bayes and Decision Tree
- NB learner creates three classifiers: $C_{e1,NB}, C_{e2,NB}, C_{e3,NB}$
 - e.g., $C_{e1,NB}$ will decide if a given data instance belongs to e_1
- To train $C_{e1,NB}$, use training sources $S_1, \dots, S_m$
 - suppose when matching these to G , we found that only two attributes a and b matches e_1
 - use data instances of a and b as positive examples
 - use data instances of other attributes of $S_1, \dots, S_m$ as negative examples
- Training other classifiers proceeds similarly
- Training meta-learner produces 6 weights:
 - $w_{e1,NB}, w_{e1,DT}, \dots, w_{e3,NB}, w_{e3,DT}$

Multi-Strategy Learning: Matching Phase

- Given a new schema S with attributes $e_1', \dots, e_n'$
- Apply learners $L_1, \dots, L_k$ to $e_1', \dots, e_n'$
- Combine the predictions of the learners
 - $p_e(e') = \sum_{i=1}^k w_{\{e, L_i\}} * p_{\{e, L_i\}}(e')$
 - $p_e(e')$: the overall sim score between attribute e of mediated schema and attribute e' (which is $e_1', e_2', \dots$, or e_n')
 - $p_{\{e, L_i\}}$: the sim score learner L_i computes between e and e'

118

Example of Matching Phase

- Recall from the example of training phase
 - G has three attributes e_1, e_2, e_3 ; two learners NB and DT
 - with classifiers $C_{e1, NB}, C_{e2, NB}, C_{e3, NB}$ and $C_{e1, DT}, C_{e2, DT}, C_{e3, DT}$
 - meta-learner has six weights $w_{e1, NB}, w_{e1, DT}, \dots, w_{e3, NB}, w_{e3, DT}$
- Let S be a new source with attributes e_1' and e_2'
- NB learner produces a 3*2 matrix of sim scores
 - $p_{e1, NB}(e_1'), p_{e1, NB}(e_2')$ [by classifier $C_{e1, NB}$]
 - $p_{e2, NB}(e_1'), p_{e2, NB}(e_2')$ [by classifier $C_{e2, NB}$]
 - $p_{e3, NB}(e_1'), p_{e3, NB}(e_2')$ [by classifier $C_{e3, NB}$]
- DT learner produces a similar sim matrix
- Meta-learner combines the predictions
 - $p_{e1}(e_1') = w_{e1, NB} * p_{e1, NB}(e_1') + w_{e1, DT} * p_{e1, DT}(e_1')$

119

Discussion

- Mapping to the generic schema matching architecture
 - learners = matchers
 - meta-learner = combiner
- Here the matchers and combiner use machine learning techniques → enable them to learn from previous matching experiences (of sources $S_1, \dots, S_m$)
- Note that even when we match just two sources S and T , we can still use machine learning techniques in the matchers and combiners
 - e.g., if the data instances of source S are available, they can be used as training data to build classifiers over S



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

120

120

Outline

- Problem definition, challenges, and overview
- Schema matching
 - Matchers
 - Combining match predictions
 - Enforcing domain integrity constraints
 - Match selector
 - Reusing previous matches
 - Many-to-many matches
- Schema mapping



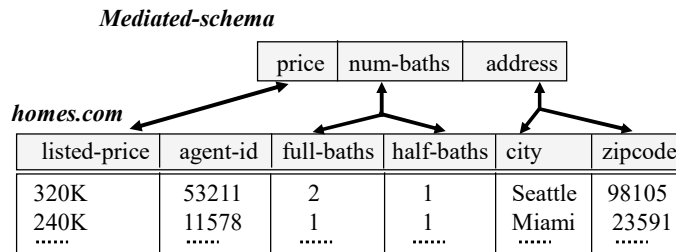
SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

121

121

Many-to-Many Matching

- Consider matches between *combinations* of columns
 - ... unlimited search space!
- Key challenge: control the search.



SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

122

Search for Complex Matches

- Employ specialized searchers:
 - Text searcher: concatenations of columns
 - Numeric searcher: arithmetic expressions
 - Date searcher: combine month/year/date
- Evaluate match candidates:
 - Compare with learned models
 - Statistics on data instances
 - Typical heuristics

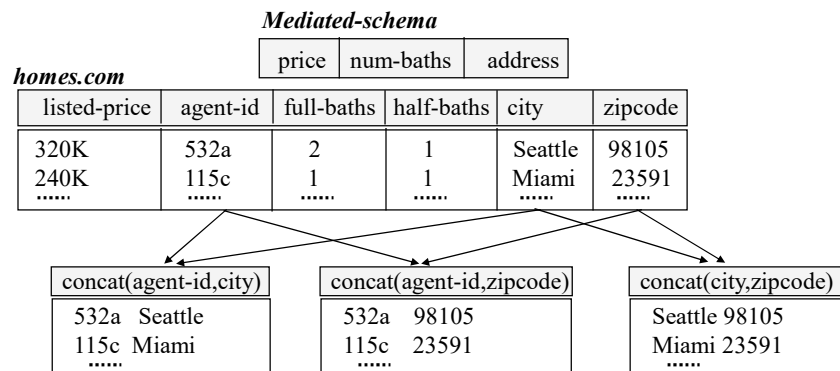


SOICT TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
School of Information and Communication Technology

123

An Example: Text Searcher

- Best match candidates for address
 - (agent-id,0.7), (concat(agent-id,city),0.75), (concat(city,zipcode),0.9)



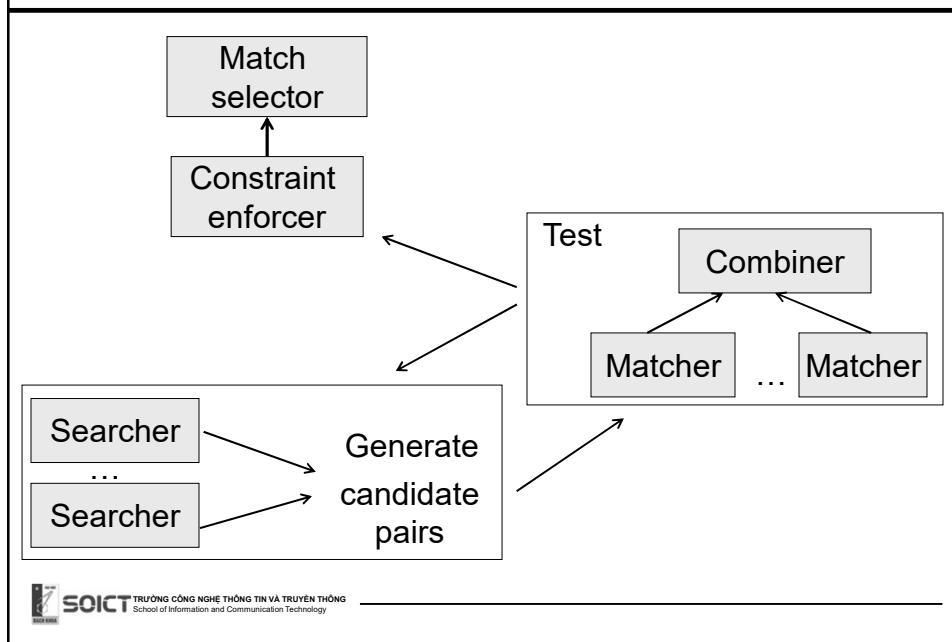
124

Controlling the Search

- Limit the search with beam search:
 - Consider only top k candidates at every level of the search
- Termination based on diminishing returns:
 - Estimate of quality does not change much between iterations
- Details of a system that did this:
 - iMap [Doan et al., SIGMOD, 2004]

125

Modified Architecture



126

Outline

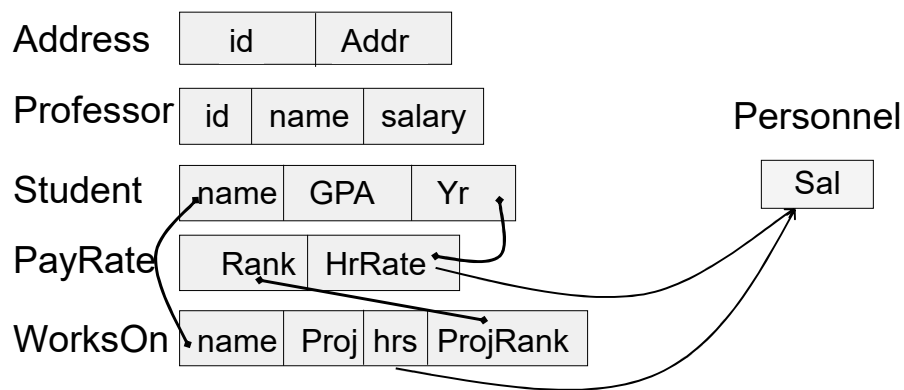
- Problem definition, challenges, and overview
- Schema matching
 - Matchers
 - Combining match predictions
 - Enforcing domain integrity constraints
 - Match selector
 - Reusing previous matches
 - Many-to-many matches
- Schema mapping

127

From Matching to Mapping

- Input:
 - Schema matches
 - Constraints (if available)
- Output:
 - Schema mappings
 - (for now, let's do SQL)
- Let's look at the choices we need to make
 - A solution will emerge...
 - Based on the IBM Clio Project

Multiple Join Paths



$$f1: \text{PayRate}(\text{HrRate}) * \text{WorksOn}(\text{Hrs}) = \text{Personnel}(\text{Sal})$$

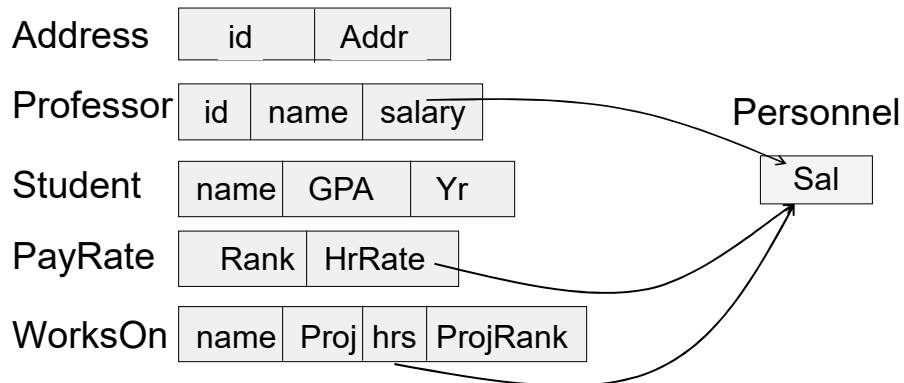
Two Possible Queries

```
select P.HrRate * W.hrs
from PayRate P, WorksOn W
where P.Rank = W.ProjRank
```

```
select P.HrRate * W.hrs
from PayRate P, WorksOn W, Student S
where W.Name=S.Name and
      S.Yr = P.Rank
```

We could also consider the Cartesian product
but that seems intuitively wrong.

Horizontal partitioning



f2: Professor(Sal) → Personnel(Sal)

What Kind of Union?

```
select P.HrRate * W.hrs  
from PayRate P, WorksOn W  
where P.Rank = W.ProjRank
```

UNION ALL

```
select Sal  
from Professor
```

Could also do an outer-union ... and even a join.

Two Sets of Decisions

- What join paths to choose?
 - (we'll call these 'candidate sets')
- How to combine the results of the joins?
- Underlying database-design principles:
 - Values in the source should appear in the target
 - They should only appear once
 - We should not lose information

Join Paths

- Discover candidate join paths by:
 - following foreign keys
 - look at paths used in queries
 - paths discovered by mining data for joinable columns.
- Select paths by:
 - Prefer foreign keys
 - Prefer ones that involve a constraint
 - Prefer smaller difference between inner and outer joins.
- Result of this step: candidate sets.

Selecting Covers

- Candidate cover: a minimal set of candidate sets that covers all the input correspondences
- Select best cover:
 - Prefer with fewest candidate paths
 - Prefer one that covers more attributes of target
- Express mapping as union of candidate sets in selected cover.

Summary

- Schema matching:
 - Use multiple matchers and combine results
 - Learn from the past
 - Incorporate constraints and user feedback
- From matching to mapping:
 - Search through possible queries
 - Principles from database design guide search
 - User interaction is key

